Offensive Comment Filtering Based on NLP

Qiqing Gao[1], Hongchi Zhou[1]

[1] Department of Applied Mathematics and Statistics, Johns Hopkins University

Abstract :

The rapid growth of online platforms has led to increasing challenges in detecting offensive and toxic language. Traditional methods often struggle to identify context-dependent abuse and assess its severity effectively. In this work, we present an offensive comment filtering system that combines multi-label classification with a toxicity scoring mechanism. Leveraging the lightweight Transformer model DistilBERT, we fine-tune it for toxicity detection, achieving a multi-label classification accuracy close to 93%. To further optimize toxicity rating, we apply three optimization algorithms—SPSA, GA, and SA—with the best approach reaching an accuracy of approximately 70%. Our system offers a practical and efficient solution for scalable content moderation. We provide replication code at `https://github.com/hliu4140/MLProject`.

Keywords: Offensive Language Detection, BERT, DistilBERT, Simultaneous Perturbation Stochastic Approximation (SPSA), Genetic Algorithm (GA), Simulated Annealing (SA).

1 Introduction

As the explosion of social platforms has brought about a surge in user content, there has been a marked increase in offensive and toxic language, such as harassment, hate speech, threats and subtle name-calling. This state of affairs undermines a healthy community discourse environment while posing psychological risks to users. Manual review can't keep up with the millions of posts per day, so there's an urgent need for automated NLP-based solutions.

Detecting toxicity is challenging because offensive content is context-sensitive and multi-dimensional: a single comment may simultaneously be obscene, insulting, or threatening. Keyword filters miss veiled insults or over-flag benign uses; traditional classifiers rely on hand-crafted features and often fail to generalize to new forms of abuse. Moreover, most systems yield only binary or categorical judgments, lacking a continuous measure of how severe or urgent a comment is—a limitation for platforms that wish to apply graduated moderation policies.

In recent years, recent advances in the field of NLP, especially Transformer-based language models, have provided promising tools to address the above challenges. Models such as BERT have shown excellent ability to capture linguistic context and semantics, outperforming previous approaches in all types of text categorization tasks. Utilizing large-scale pre-trained models and fine-tuning them for harmful speech detection tasks is expected to improve the recognition of subtly offensive language.

Despite these advancements, existing content moderation methods remain inadequate in several respects. Traditional machine learning classifiers, when applied to this task, often require extensive feature engineering and still may not generalize well to the diverse and evolving ways in which people express toxicity. Purely rule-based approaches are brittle and easily evaded by intentional misspellings or novel slang. Even modern deep learning models, while powerful, are typically used to produce categorical outputs and do not inherently solve the problem of combining multiple toxicity dimensions into a single interpretable score. Additionally, off-the-shelf models may not be optimized for the specific context of toxic comments, and they can exhibit unintended biases (e.g., over-flagging comments that contain certain identity-related keywords regardless of actual harm, a problem noted in prior studies). There is a clear need for an improved system that can more accurately detect toxic comments and assess their severity in a principled way.

To address these issues, we present an offensive comment filtering system that combines multi-label classification with a novel scoring mechanism. The proposed approach leverages NLP models and is evaluated on the aforementioned Jigsaw Rate Severity of Toxic Comment datasets and Toxic Comment Classification datasets to ensure robust performance. The key contributions of our work are summarized as follows:

2 Literature Review

2.1 Domain Background

The proliferation of user-generated content on social media has heightened concern over offensive and hateful language, creating a need for automated filtering via natural language processing (NLP) (Schmidt et al., 2017). Offensive comments encompass everything from profanity and personal insults to hate speech targeting protected groups, and such content can seriously disrupt online communities (Fortuna et al., 2018). Platforms have thus turned to automated content moderation tools to detect toxic language. The detection task is typically framed as a text classification problem, identifying whether a comment is offensive and often categorizing its subtype or target (Zampieri et al., 2019). In support of this, large annotated datasets of toxic comments have been released. For example, the Jigsaw Toxic Comment Classification contains hundreds of thousands of online comments labeled with multiple toxicity categories (Jigsaw, 2018). These resources provide a solid foundation for developing NLP models to filter or score user comments based on offensiveness.

2.2 Related Work

Early efforts in automated offensive language detection relied on lexical methods such as keyword blacklists or simple rule-based filters. These approaches could flag obvious profanity but often failed to account for linguistic variation and context, leading to many false positives (Dixon et al., 2018). Supervised machine learning methods subsequently improved performance: classifiers like support vector machines or logistic regression, trained on n-gram and frequency-based features, learned to detect hate speech

more robustly than naive keyword matching (Waseem et al., 2016; Davidson et al., 2017). However, these models still relied heavily on surface text cues and lacked deeper semantic understanding.

Deep neural network models brought further improvements by automatically extracting and combining features from text. Architectures such as convolutional or recurrent neural networks showed success in learning the subtleties of offensive language from word embeddings (Badjatiya et al., 2017). Still, training such networks from scratch demands substantial labeled data, which is often limited in this domain (Fortuna et al., 2018). This data bottleneck was overcome by transfer learning with large pre-trained language models.

A major advance came with transformer-based models (Vaswani et al., 2017). BERT (Bidirectional Encoder Representations from Transformers) provides deep bidirectional language representations learned from massive corpora (Devlin et al., 2019). Fine-tuning BERT on toxic comment datasets has set new state-of-the-art results in hate speech and toxic comment classification (Mozafari et al., 2020). Building on BERT, domain-specific variants have been introduced to handle abusive content. For example, HateBERT, a BERT model retrained on posts from banned hate communities, better handles the slang and coded language of those forums, improving hate speech detection (Caselli et al., 2021). In parallel, model compression has addressed BERT's deployment challenges: DistilBERT retains most of BERT's accuracy with around 40% fewer parameters, enabling much faster inference (Sanh et al., 2019). Such efficient transformer models make real-time toxic comment filtering more feasible.

2.3 Limitations of Existing Methods

Despite these advances, current approaches still face several limitations. A key issue is the context dependence of offensive language – the same phrase may be harmless or harmful depending on context and intent, yet most classifiers consider only the isolated comment text (Sap et al., 2019). This makes it difficult for models to detect implicit hate speech, sarcasm, or veiled insults that require background knowledge to identify (ElSherief et al., 2021). Moreover, over-reliance on keywords can yield false positives: benign mentions of certain demographic groups are often misclassified as hate speech when models naively associate those terms with toxicity (Dixon et al., 2018). Another concern is data imbalance and bias. Toxic comments are relatively rare, so without careful handling, classifiers become biased toward the majority (non-toxic) class and may under-detect more subtle abuse (Fortuna et al., 2018). Biases in training data or labeling practices can also propagate into models, resulting in unfair or inconsistent outcomes (Sap et al., 2019).

Furthermore, most automated systems output only categorical labels (e.g., "toxic" or specific categories) rather than a graded severity measure. This lack of a continuous toxicity metric hinders the ability to prioritize content by harm level. Finally, the computational cost of state-of-the-art models poses practical limitations: processing millions of comments in real time is often infeasible with such resource-intensive classifiers, creating a need for more efficient solutions (Sanh et al., 2019).

2.4 Innovation of This Project

Our project builds on these advancements and addresses these limitations with a novel approach. We propose a two-step inferencing framework that produces a nuanced toxicity score instead of just a class label. In the first step, we choose model that outputs probabilities for multiple toxicity categories (e.g., insult, threat, hate speech) per comment based on model performance and efficiency. In the second step, we aggregate these predictions into a single continuous score of overall offensiveness by applying optimized weights to each category. We determine the weights using multi-criteria decision techniques so that the composite score aligns with human judgments of severity. This scoring mechanism goes beyond standard classification – rather than treating all flagged comments equally, it differentiates them by degree of toxicity. Moderators can thus rank comments by their score and prioritize the most harmful content for review.

3 Problem Setup

Offensive comment detection in user-generated content has become a critical task in natural language processing (NLP), with applications ranging from content moderation to user behavior analysis. However, this task remains challenging due to several key limitations in existing approaches.

First, traditional machine learning methods often depend on some manually engineered features like TF-IDF vectors, etc. Although these approaches make computation efficient, they typically fail to capture the nuanced linguistic patterns and contextual dependencies that characterize toxic language. Therefore, their predictive performance is often limited, especially in scenarios involving sarcasm, implicit threats, etc.

Second, although large language models such as BERT have significantly advanced many NLP tasks, they are not always ideal for downstream tasks like toxicity classification. BERT is computationally expensive, making it unsuitable for deployment in real-time or resource-constrained environments. Moreover, for tasks in different fields, Bert needs to debug its general parameters in a targeted manner, otherwise it will lead to poor model performance. Therefore, there is a growing need for smaller, more specialized models that can deliver competitive performance while maintaining efficiency and adaptability to the unique characteristics of toxic language.

Third, a existing challenge lies in the lack of rating systems to quantify the level or intensity of toxicity in a given piece of text. Most existing datasets provide only binary labels (e.g., toxic vs. non-toxic), which limits the ability of models to capture the level of harmful content. This lack also restricts the utility of these models in applications where differentiating between mildly inappropriate and highly offensive content is crucial.

Therefore, to address these issues, we propose a lightweight, task-specific model designed for efficient and accurate toxicity detection. In addition, we introduce a toxicity rating model that captures distinctions in harmful content, enabling more nuanced classification and better support for downstream moderation tasks.

4 Dataset Description and Pre-processing

Our datasets are the *Jigsaw Toxic Comment Classification Challenge* (Jigsaw, 2018) and the *Jigsaw Toxic Severity Rating* (Jigsaw, 2021), both from Kaggle.

4.1 Classification Dataset

Jigsaw Toxic Comment Classification Challenge (Jigsaw, 2018) dataset comprises four files: train, test, sample submission, and test labels. In our work, we primarily use the train dataset. The train set contains 159572 comments, each annotated with six binary toxicity types: toxic, severe toxic, obscene, dangerous, insult, and identity hate. These types capture different aspects of the offensiveness of a comment. In practice, these six attributes are not completely independent, as a single comment can exhibit multiple types of toxicity simultaneously. The first 5 comments of 'train' dataset can be seen in Figure 1.

	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	0000997932d777bf	Explanation\nWhy the edits made under my usern...	0	0	0	0	0	0
1	000103f0d9cfb60f	D'aww! He matches this background colour I'm s...	0	0	0	0	0	0
2	000113f07ec002fd	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0	0
3	0001b41b1c6bb37e	"\nMore\nI can't make any real suggestions on ...	0	0	0	0	0	0
4	0001d958c54c6e35	You, sir, are my hero. Any chance you remember...	0	0	0	0	0	0

Figure 1. First 5 comments of 'train' dataset

4.2 Toxicity Rating Dataset

Jigsaw Toxic Severity Rating (Jigsaw, 2021) consists of three files, with our focus mainly on the 'validation' dataset. The validation set includes 60218 comments grouped into 30109 pairs. Each pair has been manually reviewed and labeled, designating one comment as less toxic and the other as more toxic. The first 5 comments of 'validation' dataset can be seen in Figure 2.

	worker	less_toxic	more_toxic
0	313	This article sucks \n\nwoo woo wooooooo	WHAT!!!!!!!?!?!!?!!?!?!?!?!!!!!!!!!!!!!...
1	188	"And yes, people should recognize that but the...	Daphne Guinness \n\nTop of the mornin' my fav...
2	82	Western Media?\n\nYup, because every crime in...	"Atom you don't believe actual photos of mastu...
3	347	And you removed it! You numbskull! I don't car...	You seem to have sand in your vagina.\n\nMight...
4	539	smelly vagina \n\nBluerasberry why don't you ...	hey \n\nway to support nazis, you racist

Figure 2. First 5 comments of 'validation' dataset

5 Methodology

As our model consists of two distinct components, classification and toxicity rating, we introduce them separately in the following sections.

5.1 Classification

5.1.1 BERT We utilize BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019), a transformer-based language model that has excellent performance in natural language processing tasks. BERT is based on the transformer encoder architecture, consisting of multiple layers of multi-head self-attention and feed-forward neural networks. A key innovation of BERT is its use of bidirectional self-attention, which allows the model to jointly attend to context from both the left and the right of each token, thereby capturing richer syntactic and semantic dependencies than unidirectional models.

BERT is pre-trained using two unsupervised tasks: (1) Masked Language Modeling (MLM), where random tokens in the input are masked and the model is trained to predict them based on the surrounding context, and (2) Next Sentence Prediction (NSP), which trains the model to understand sentence relationships by predicting whether two sentences appear consecutively in the corpus.

In our implementation, we utilize $\textbf{BERT}_{\text{BASE}}$, which consists of 12 transformer layers, 768 hidden dimensions, and 12 attention heads. The model is fine-tuned on our task-specific dataset by adding a task-specific classification head on top of the [CLS] token representation. During fine-tuning, all parameters of the BERT model are updated by backpropagation. The input text is first tokenized using WordPiece and padded/truncated to a fixed length. The final [CLS] token embedding is passed through a linear layer followed by a softmax activation to produce class probabilities. The overall BERT structure including pre-training and fine-tuning can be seen in Figure 3. A visualization of tokenization can be seen in Figure 4.

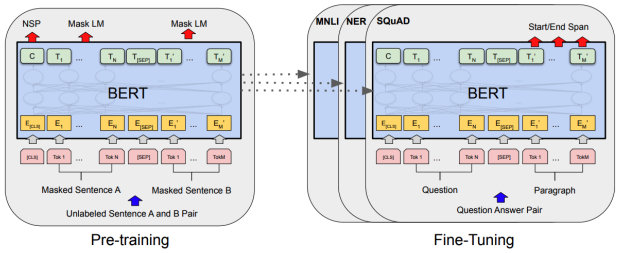

Figure 3. Overall pre-training and fine-tuning procedures for BERT. (Devlin et al., 2019)

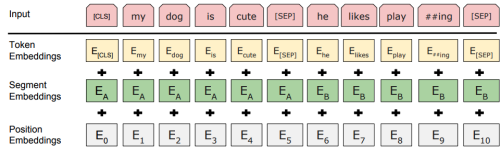

Figure 4. BERT input representation. (Devlin et al., 2019)

5.1.2 DistilBERT Due to the high computational cost and memory requirements of BERT, especially in large-scale or resource-constrained applications, we consider adopting a more lightweight alternative. One such model is DistilBERT (Sanh et al., 2019), which significantly reduces the number of parameters and the inference time while preserving most of the performance of BERT.

DistilBERT is trained using a knowledge distillation framework, where a compact 'student' model learns to approximate the behavior of the larger 'teacher' model (BERT-base). This approach enables DistilBERT to achieve 97% of BERT's performance with only 40% of its parameters and a 60% increase in speed(Sanh et al., 2019).

Structurally, DistilBERT follows the BERT design but simplifies it in several ways: (1) The number of layers is reduced from 12 to 6; (2) Token-type embeddings and the pooler layer are removed, eliminating specific components and simplifying the architecture; (3) Parameter initialization is strategically optimized by initializing the student network with every other layer from the teacher, preserving critical feature extraction capabilities while reducing redundancy; (4) Computational efficiency is prioritized by focusing on reducing transformer layers rather than hidden dimensions. The model architecture of DistilBERT can be seen in Figure 5.

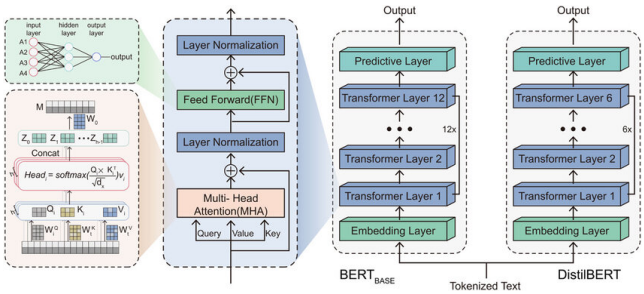

Figure 5. Schematic diagram of BERT-BASE and DistilBERT model architecture (Li et al., 2024)

5.2 Toxicity Rating

Based on the classification results from DistilBERT, we can build a toxicity rating model as

$$G(X_k, w) = \sum_{i=1}^{6} w_i x_i \tag{1}$$

where $X_k = (x_k^{(1)}, \cdots, x_k^{(6)})$, with each $x_k^{(i)} \in [0,1], i = 1, \cdots, 6$ represents the predicted probabilities for the k-th comment on six classification labels: toxic, severe toxic, obscene, threat, insult, and identity hate; $w = (w_1, \cdots, w_6) \in \mathcal{W} = \{Z \in [0,1]^6 | \sum_{i=1}^{6} z_i = 1\}$ is the weight vector of the 6 labels above. The value of the function G represents the score of the toxicity of the comment. The higher the score, the greater the toxicity of the comment.

Based on the 'validation' dataset described in the previous section, we denote the less toxic comments as l and the more toxic comments as m. Determining the optimal weight vector thus becomes an optimization problem, where the objective function is defined as follows:

$$\min_{w \in \mathcal{W}} L(w) = \min_{w \in \mathcal{W}} 1 - \frac{1}{N} \sum_{k=1}^{N} \mathbf{1}_{G(m_k, W) > G(l_k, W)} \tag{2}$$

where $\mathbf{1}_{G(m_k,W)>G(l_k,W)}$ is the indicator function that returns 1 if $G(m_k, W) > G(l_k, W)$, and 0 otherwise. To solve this optimization problem, we consider the following optimization algorithms.

5.2.1 Simultaneous Perturbation Stochastic Approximation (SPSA)

To further improve the efficiency and robustness of the weight optimization process, we employ the Simultaneous Perturbation Stochastic Approximation (SPSA) algorithm (Spall, 1992).

The Simultaneous Perturbation Stochastic Approximation (SPSA) algorithm is used to minimize an objective function $L(w)$ when the explicit gradient is not available or too expensive to compute. The method approximates the gradient by perturbing all components of the parameter vector simultaneously with a small random vector.

At iteration t, let $\hat{w}_t \in \mathcal{W}$ be the current estimate of the optimal weight vector. Then the stochastic gradient is defined as

$$\hat{g}_t^{(i)}(\hat{w}_t) = \frac{L(\hat{w}_t + c_k \Delta_t) - L(\hat{w}_t - c_k \Delta_t)}{2 c_k \Delta_t^{(i)}} \tag{3}$$

where $\Delta_t = (\Delta_t^{(1)}, \cdots, \Delta_t^{(6)}) \in \mathbb{R}^6$ is a random perturbation vector, and each component is i.i.d, drawn from a Bernoulli(± 1), $c_k > 0$ is a small positive scalar. The parameter update is then performed via

$$\hat{w}_{t+1} = \hat{w}_t - a_t \hat{g}_t(\hat{w}_t) \tag{4}$$

where $a_t > 0$ is the step size at iteration t.

In practice, to ensure stability and convergence of the SPSA algorithm, the sequences $\{a_t\}$ and $\{c_t\}$ are typically chosen according to the following forms:

$$a_t = \frac{a}{(A + t + 1)^{\alpha}} \quad c_t = \frac{c}{(k+1)^{\gamma}} \tag{5}$$

where $a, c, A, \alpha, \gamma > 0$ are tuning parameters. For the practical implementation of the SPSA algorithm, the tuning parameters are chosen as $a = 0.5, A = 50, c = 0.1, \alpha = 0.602$,and $\gamma = 0.101$, following common recommendations to ensure stability and convergence (Spall, 2003, p. 190). The pseudocode of SPSA is presented as follows:

Algorithm 1 Simultaneous Perturbation Stochastic Approximation (SPSA)

Initialize: $k \leftarrow 0$, choose initial guess $\hat{w}_0$, coefficients a, c, A, α, γ
Set: $a_t = \frac{a}{(A+t+1)^{\alpha}}, \quad c_t = \frac{c}{(t+1)^{\gamma}}$
while stopping criterion not met **do**
  Generate $\Delta_t \in \mathbb{R}^6$ with each component $\Delta_t^{(i)} \sim$ Bernoulli(± 1)
  Evaluate: $L^+ = L(\hat{w}_t + c_t \Delta_t)$, $L^- = L(\hat{w}_t - c_t \Delta_t)$
  Estimate gradient:

$$\hat{g}_t = \frac{L^+ - L^-}{2 c_t} \cdot \Delta_t^{-1} \quad \text{(element-wise division)}$$

  Update parameter: $\hat{w}_{t+1} = \hat{w}_t - a_t \hat{g}_t$
  $t \leftarrow t + 1$
end while
return $\hat{w}_t$

5.2.2 Genetic Algorithm (GA)

In addition to the previously mentioned methods, we also considered another optimization algorithm — the Genetic Algorithm (GA)(Holland, 1975). Inspired by the principles of natural selection and genetics, GA belongs to the class of evolutionary algorithms and is particularly well-suited for solving complex optimization problems in high-dimensional, non-convex spaces.

For the realization of the algorithm, we begin by randomly generating the initial population $\mathcal{P}^{(0)} = \{w_{0,1}, \ldots, w_{0,N}\}$, where each individual $w_{0,i} \in \mathcal{W}$, and we set $N = 100$ in our implementation. At each generation t, we evaluate the loss function $L(w_{t,i})$ for each individual and denote the corresponding fitness as $F_{t,i} = L(w_{t,i})$.

For selection, the top $\rho = 30\%$ of individuals with the lowest loss are chosen as parents, and the top 5% are directly carried over to the next generation via an elitism strategy. Crossover is then applied to the selected parents to generate offspring. Specifically, we use uniform crossover: for each gene $j = 1, \ldots, 6$, the child inherits the gene from one of the two parents with equal probability:

$$\text{Child}^{(j)} = \begin{cases} w_1^{(j)}, & \text{with probability } 0.5 \\ w_2^{(j)}, & \text{with probability } 0.5 \end{cases} \tag{6}$$

where $\text{Parent}_i = (w_i^{(1)}, \ldots, w_i^{(6)})$ for $i = 1, 2$, and $\text{Child} \in \mathcal{W}$ denotes the resulting offspring.

Mutation is then applied to each gene of the offspring with probability $p_{\text{mut}} = 0.1$ to enhance diversity and prevent premature convergence:

$$w_j^{\text{mut}} = \begin{cases} \text{Perturb}(w^{\text{child}}j), & \text{with probability } p_{\text{mut}} \\ w_j^{\text{child}}, & \text{otherwise} \end{cases} \tag{7}$$

The new generation $\mathcal{P}^{(t+1)}$ is formed by combining the mutated offspring with the elite individuals. The algorithm terminates early if no improvement in the best fitness is observed over 20 consecutive generations. The pseudocode of GA is presented as follows:

Algorithm 2 Genetic Algorithm for Weight Optimization

Initialize population $\mathcal{P}^{(0)} = \{w_{0,1}, \ldots, w_{0,N}\}$ with $w_{0,i} \in \mathcal{W}$
for $t = 1$ to max_generation **do**
  Evaluate fitness: $F_{t,i} = L(w_{t,i})$
  Select top ρ% as parents, keep top 5% as elites
  while new population size $< N$ **do**
    Randomly select two parents from parent pool
    Apply uniform crossover with probability p_{cross}
    Mutate offspring with probability p_{mut}
    Add offspring to new population
  end while
  Replace current population with new population and elites
  if no improvement in best $L(w)$ for 20 generations **then**
    break
  end if
end for
Normalize: $\hat{w} = \frac{w^*}{\sum_{i=1}^{p} (w^*)^{(i)}}$
return $\hat{w}$

5.2.3 Simulated Annealing (SA) In addition to the previously mentioned methods, we also considered Simulated Annealing (SA) (Kirkpatrick et al., 1983). Inspired by the annealing process in metallurgy, SA is an iterative probabilistic algorithm designed to escape local optima and find global optima in complex, high-dimensional spaces, making it suitable for solving difficult optimization problems.

The goal of the algorithm is still to find the optimal weight vector $w = (w^{(1)}, \cdots, w^{(6)})$ that minimizes our loss function $L(w)$. For the realization of the algorithm, we start with a random initial guess for the weight $w_0 \in \mathcal{W}$ and also set the initial temperature T_0, cooling rate α. Then at each iteration, we generate a neighboring solution w_t' by perturbing the current weight vector w_t. The perturbation is typically a small random change within the predefined bounds, ensuring that the new weight vector remains within the valid range.

And we define our acceptance criteria as following

- if $L(w_t') < L(w_t)$, $w_{t+1} = w_t'$
- if $L(w_t') \geq L(w_t)$

$$w_{t+1} = \begin{cases} w_t', & \text{with probability } P = \exp\left(\frac{L(w_t) - L(w_t')}{T_t}\right) \\ w_t, & \text{with probability } 1 - P \end{cases}$$

where $T_{t+1} = \alpha T_t, t \in \{0, 1, \cdots\}$. The pseudocode of SA is presented as follows:

Algorithm 3 Simulated Annealing for Loss Minimization

Input: Initial weight vector $\mathbf{w}_0$, initial temperature T_0, cooling rate α, loss function $\mathcal{L}(\mathbf{w})$, maximum iterations $k_{\max}$
Output: Optimal weight vector $\mathbf{w}^*$, optimal loss $\mathcal{L}(\mathbf{w}^*)$
Initialize $\mathbf{w}_0$ randomly, clipped to $[0, 1]$ and normalized such that $\sum \mathbf{w}_0 = 1$
Set T_0, α, and $T_{\min}$ (minimum temperature)
$k \leftarrow 0$, $\mathcal{L}_0 \leftarrow \mathcal{L}(\mathbf{w}_0)$
while $k \leq k_{\max}$ and $T_k > T_{\min}$ **do**
  Generate neighboring solution $\mathbf{w}_k'$ by perturbing $\mathbf{w}_k$
  Clip and normalize $\mathbf{w}_k'$ to satisfy $0 \leq w_i' \leq 1$ and $\sum w_i' = 1$
  Compute the loss $\mathcal{L}(\mathbf{w}_k')$
  if $\mathcal{L}(\mathbf{w}_k') < \mathcal{L}(\mathbf{w}_k)$ **then**
    Accept $\mathbf{w}_k'$ as the new solution: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k'$
  else
    Compute acceptance probability:

$$P = \exp\left(\frac{\mathcal{L}(\mathbf{w}_k) - \mathcal{L}(\mathbf{w}_k')}{T_k}\right)$$

    if $P \geq$ random number **then**
      Accept $\mathbf{w}_k'$ as the new solution: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k'$
    else
      Retain current solution: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k$
    end if
  end if
  Update temperature: $T_{k+1} \leftarrow \alpha T_k$
  Increment iteration: $k \leftarrow k + 1$
end while
Return: $\mathbf{w}^* = \mathbf{w}_k$, $\mathcal{L}(\mathbf{w}^*)$

6 Experiment

In this section, we demonstrate our method in 2 steps. In the first step, we provided fine-tuned large models to produce probabilities of toxicity. In the second step, we apply the algorithms stated in previous sections to measure the ratings of toxicity of comments.

6.1 Model Choosing

To complete the stage one for evaluating the degree of toxicity, we fine-tune DistilBERT, one of the state of art large Natural Language Processing Model to produce probability of whether one specific sentence belongs to some types of toxicity. After that, we add classification head on top of the model to predict the probabilities that

could be utilized in stage 2, the part for assessing the degree of toxicity for sentences.

6.1.1 Dataset1 We use a dataset from Kaggle's Toxic Comment Classification Challenge(Jigsaw, 2018), which contained Wikipedia comments which have been labeled by human raters. Toxicity type includes "toxic", "severe toxic", "obscene","threat","insult", and "identity hate". This dataset is suitable for our stage 1 calculation.

6.1.2 Model Comparison We use 2 models as benchmark, BERT and HateBERT(Caselli et al., 2021), to compare with the performance of DistilBERT. We employ DistilBERT model as our baseline for the stage 1 calculation since the training time of Distill BERT outperforms the others. Just as what we described in previous methodology section, DistilBERT has fewer neurons than traditional BERT model, and fewer times for the model to converge. So we run experiments comparing the Traditional BERT, HateBERT, a variant of BERT model that pretrained on RAL-E, a large-scale dataset of Reddit comments in English from communities banned for being offensive, abusive, or hateful that we have collected and made available to the public(Caselli et al., 2021). We run each model that we choose with 10 epochs, with learning rate as 3e-5, batch size 64, and maximum length of sentences 384. We considered the data set as 90% for training and 10% for testing. After running 10 epochs, we see that although Distill BERT sacrifices some accuracies,it requires training time that only half of the other models for each epoch. The comparison results can be seen in Figure 6.

Therefore, compared to BERT and HateBERT, DistilBERT offers greater practical value in real-world scenarios. It achieves comparable performance with only a slight increase in the loss function, while significantly reducing computational cost and improving inference speed.

6.2 Classification

6.2.1 Dataset2 In the second stage of our experiment, we utilized the dataset 'validation' provided by the Jigsaw Toxic Comment Severity Rating competition on Kaggle (Jigsaw, 2021), which contains paired comments with relative toxicity labels — one being labeled as more toxic and the other as less toxic. This structure is particularly suitable for training models to learn comparative toxicity.

6.2.2 Model Results After fine-tuning the pre-trained DistilBERT model and training it on the 'validation dataset', we obtained the predicted probabilities for both less toxic and more toxic comments across six types of toxicity. The first 3 predicted probabilities for both less toxic and more toxic comments can be seen in Figure 7 and Figure 8.

toxic	severe_toxic	obscene	threat	insult	identity_hate
0.991691	0.027857	0.931590	0.001045	0.452971	0.002090
0.016341	0.000037	0.000237	0.000056	0.000858	0.000120
0.044445	0.000083	0.000165	0.000070	0.000999	0.001721

Figure 7. Predicted probabilities of first 3 less toxic comments

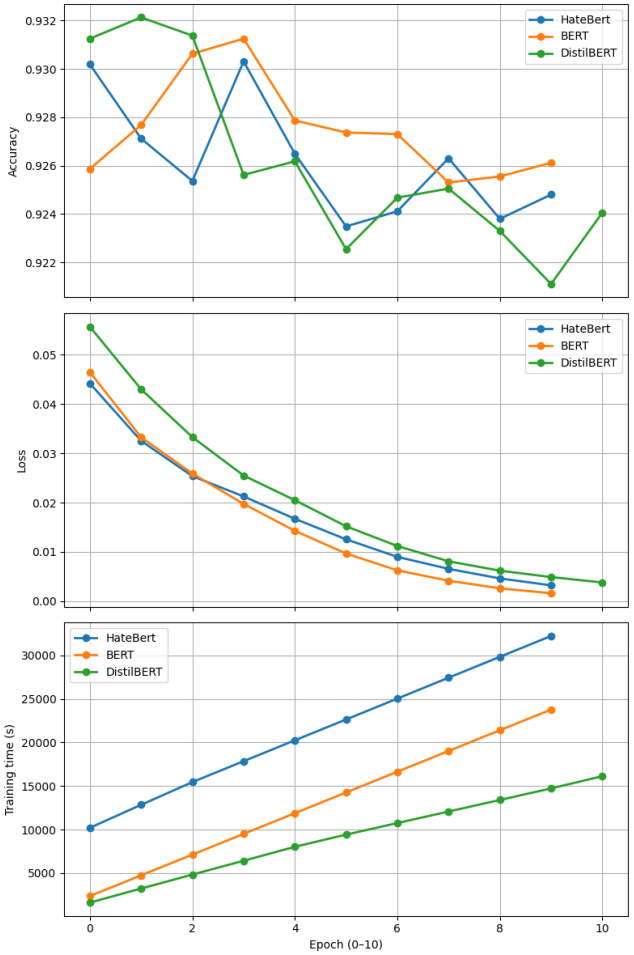

Figure 6. Performance Measure Comparison between BERT vs HateBERT vs DistillBERT

toxic	severe_toxic	obscene	threat	insult	identity_hate
0.751445	0.001601	0.023274	0.000133	0.013448	0.001426
0.717482	0.004181	0.128471	0.003781	0.293542	0.011043
0.028851	0.000048	0.002542	0.000046	0.000962	0.000104

Figure 8. Predicted probabilities of first 3 more toxic comments

6.3 Toxicity Rating

Based on the prediction results from the classification stage and the objective function defined earlier, we applied SPSA, Genetic Algorithm (GA), and Simulated Annealing (SA) to optimize the loss function. To ensure the reproducibility of our results, we set the random seed to '66' for all optimization procedures. The optimal weight vectors obtained by each method are presented in Table 1.

Algorithm \ Weight	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	$w^{(4)}$	$w^{(5)}$	$w^{(6)}$	$L(w)$
SPSA	0.3066	0.3364	0.0890	0.1127	0.0047	0.1506	0.3012
GA	0.4674	0.3307	0.0568	0.0150	0.0211	0.1090	0.2993
SA	0.3906	0.5328	0.0007	0.0137	0.0252	0.0370	0.3000

Table 1
Optimized weight vectors and corresponding loss values for each algorithm.

A comparison of the three algorithms in terms of loss value versus iteration is illustrated in Figure 9 and Figure 10.

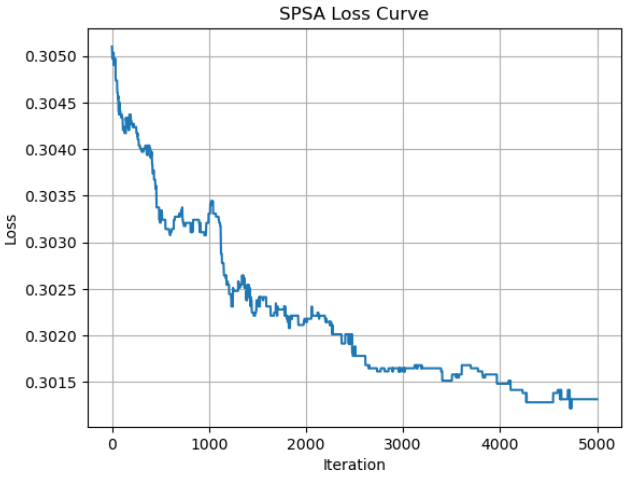

Figure 9. Loss curve of SPSA

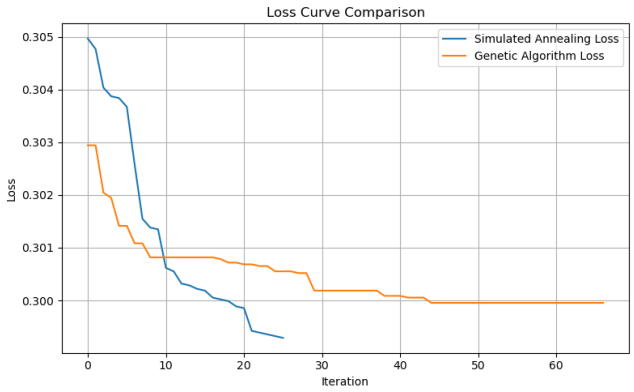

Figure 10. Loss curve comparison between GA and SA

Based on the comparison of the loss curves, we observe that the optimal solutions obtained by different optimization algorithms are highly similar. However, in real-world applications, the efficiency of completing the rating task is also an important factor to consider. In our implementation, the running times for SPSA, GA, and SA were 4 minutes 53.2 seconds, 3.87 seconds and 2.03 seconds, respectively. Therefore, considering both solution quality and computational efficiency, we conclude that SA achieves the highest efficiency and adopt it as the final algorithm to implement the toxicity rating task.

7 Conclusion

In this project, we proposed a two-stage approach for toxicity severity rating. In the first stage, we fine-tuned the pre-trained DistilBERT model to predict the probabilities of six types of toxicity for given comments. Compared to traditional BERT and HateBERT models, DistilBERT achieved comparable performance with significantly lower computational costs and faster inference times, making it more practical for real-world applications.

In the second stage, based on the predicted probabilities and a well-defined objective function, we applied three optimization algorithms—SPSA, GA, and SA—to determine the optimal weight vector for toxicity rating. Experimental results showed that while all three algorithms converged to highly similar optimal solutions in terms of loss value, Simulated Annealing (SA) demonstrated the highest efficiency with the shortest running time. Therefore, SA was selected as the final optimization method for the toxicity rating task.

Overall, our experiments highlight that lightweight models combined with efficient optimization algorithms can provide effective and scalable solutions for real-world toxicity rating challenges.

8 Further Work

While our proposed framework achieves promising results in toxicity severity rating, there are several directions for future improvement.

First, although DistilBERT provides a good balance between computation efficiency and model performance, exploring other lightweight transformer models such as TinyBERT (Jiao et al., 2020) or MiniLM (Wang et al., 2020) could further enhance model efficiency, especially for large-scale scenarios.

Second, in our optimization stage, we use standard SPSA, GA and SA. Future work could explore more advanced algorithms, like SPSA based on CRN (Spall, 2003), combining GA and SA, to achieve faster convergence and better robustness against local minimum.

Third, cultural and linguistic differences can greatly impact how toxic language is expressed. In practice, users often attempt to evade detection by replacing certain toxic words with alternative characters such as “1”, “@”, or “!”. This phenomena is called ’Algospeak’ (Steen et al., 2023). Future work could involve designing or utilizing robust text pre-processing methods or adversarial training techniques like BAE (Garg et al., 2020) to handle these disguised toxic expressions, thereby improving the model’s robustness and generalizability across diverse cultural contexts.

Finally, validating our approach on more diverse datasets, including multilingual or domain-specific toxicity datasets, would help evaluate the generalizability of our method and explore potential for domain adaptation.

Overall, these directions offer promising opportunities for building a more efficient, accurate, and robust toxicity severity rating system.

References

Badjatiya, Pratik et al. (2017). “Deep Learning for Hate Speech Detection in Tweets”. In: *Proceedings of the 26th International Conference on World Wide Web Companion (WWW ’17 Companion)*, pp. 759–760. DOI: 10.1145/3041021.3054224.

Caselli, Tommaso et al. (2021). “HateBERT: Retraining BERT for Abusive Language Detection in English”. In: *Proceedings of the 5th Workshop on Online Abuse and Harms (WOAH 2021)*. Association for Computational Linguistics, pp. 17–25. DOI: 10.18653/v1/2021.woah-1.3. URL: https://aclanthology.org/2021.woah-1.3/.

Davidson, Thomas et al. (2017). “Automated Hate Speech Detection and the Problem of Offensive Language”. In: *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM ’17)*, pp. 512–515.

Devlin, Jacob et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

Dixon, Lucas et al. (2018). “Measuring and Mitigating Unintended Bias in Text Classification”. In: *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, pp. 67–73. DOI: 10.1145/3278721.3278727.

ElSherief, Mai et al. (2021). “Latent Hatred: A Benchmark for Understanding Implicit Hate Speech”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 345–363. DOI: 10.18653/v1/2021.emnlp-main.29. URL: https://aclanthology.org/2021.emnlp-main.29/.

Fortuna, P. and S. Nunes (2018). “A Survey on Automated Detection of Hate Speech in Text”. In: *ACM Computing Surveys* 51.4, 85:1–85:30. DOI: 10.1145/3232676.

Garg, Siddhant and Goutham Ramakrishnan (2020). “BAE: BERT-based Adversarial Examples for Text Classification”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics. DOI: 10.18653/v1/2020.emnlp-main.498. URL: http://dx.doi.org/10.18653/v1/2020.emnlp-main.498.

Holland, John H. (1975). *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press.

Jiao, Xiaoqi et al. (2020). *TinyBERT: Distilling BERT for Natural Language Understanding*. arXiv: 1909.10351 [cs.CL]. URL: https://arxiv.org/abs/1909.10351.

Jigsaw (2018). *Jigsaw Toxic Comment Classification Challenge*. Kaggle dataset. Accessed: February 2025. URL: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge/data.

— (2021). *Jigsaw Toxic Comment Classification Challenge*. https://www.kaggle.com/c/jigsaw-toxic-severity-rating/data. Accessed: Februray 2025.

Kirkpatrick, S., C. D. Gelatt, and M. P. Vecchi (1983). “Optimization by Simulated Annealing”. In: *Science* 220.4598, pp. 671–680. DOI: 10.1126/science.220.4598.671. eprint: https://www.science.org/doi/pdf/10.1126/science.220.4598.671. URL: https://www.science.org/doi/abs/10.1126/science.220.4598.671.

Li, Jun, Yuchen Zhu, and Kexue Sun (Aug. 2024). “A novel iteration scheme with conjugate gradient for faster pruning on transformer models”. In: *Complex Intelligent Systems* 10, pp. 1–13. DOI: 10.1007/s40747-024-01595-w.

Mozafari, Milad, Reza Farahbakhsh, and Nicola Crespi (2020). “A BERT-based Transfer Learning Approach for Hate Speech Detection in Online Social Media”. In: *Proceedings of the Fifth International Workshop on Online Abuse and Harms (WOAH ’20)*, pp. 138–144. DOI: 10.18653/v1/2020.woah-1.14.

Sanh, Victor et al. (2019). “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”. In: *arXiv preprint arXiv:1910.01108*. URL: https://arxiv.org/abs/1910.01108.

Sap, Maarten et al. (2019). “The Risk of Racial Bias in Hate Speech Detection”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1668–1678. DOI: 10.18653/v1/P19-1164.

Schmidt, Anna and Michael Wiegand (2017). “A Survey on Hate Speech Detection Using Natural Language Processing”. In: *Proceedings of the Fifth International Workshop on NLP for Social Media (SocialNLP@ACL)*, pp. 1–10. DOI: 10.18653/v1/W17-1101.

Spall, James C. (1992). “Multivariate Stochastic Approximation Using Simultaneous Perturbation”. In: *IEEE Transactions on Automatic Control* 37.3, pp. 332–341. DOI: 10.1109/9.121804.

— (2003). *Introduction to Stochastic Search and Optimization: Estimation, Simulation, and Control*. Wiley-Interscience Series in Discrete Mathematics and Optimization. Hoboken, N.J.: Wiley-Interscience. ISBN: 978-0-471-33052-3.

Steen, Ella, Kathryn Yurechko, and Daniel Klug (2023). “You Can (Not) Say What You Want: Using Algospeak to Contest and Evade Algorithmic Content Moderation on TikTok”. In: *Social Media + Society* 9.3, p. 20563051231194586. DOI: 10.1177/20563051231194586. eprint: https://doi.org/10.1177/20563051231194586. URL: https://doi.org/10.1177/20563051231194586.

Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008. DOI: 10.5555/3295222.3295349.

Wang, Wenhui et al. (2020). *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. arXiv: 2002.10957 [cs.CL]. URL: https://arxiv.org/abs/2002.10957.

Waseem, Zeerak and Dirk Hovy (2016). “Hateful or Offensive? Automatic Classification of Web Harassment”. In: *Proceedings of the NAACL Student Research Workshop*, pp. 89–94. DOI: 10.18653/v1/N16-2014.

Zampieri, Marcos et al. (2019). “Predicting the Type and Target of Offensive Posts in Social Media”. In: *Proceedings of the 2019 Conference of the North American Chapter of the ACL (NAACL-HLT)*, pp. 1415–1420. DOI: 10.18653/v1/N19-1141.